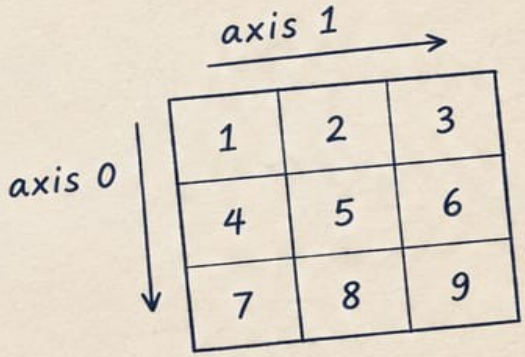
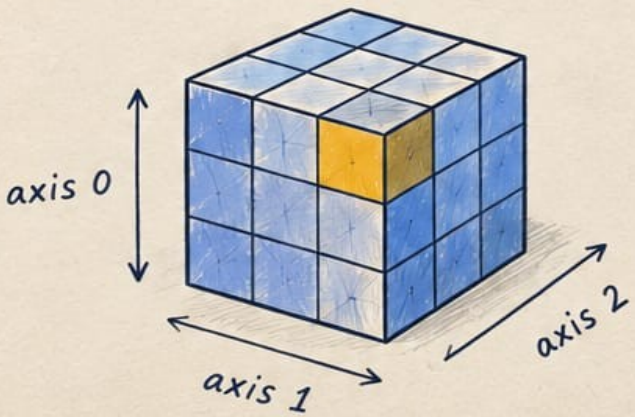
Python & NumPy Basics

```python
import numpy as np
arr = np.arange(1, 10)
matrix = arr.reshape(3, 3)
print(matrix)
```





THE BASICS OF

By Milan C. Soni

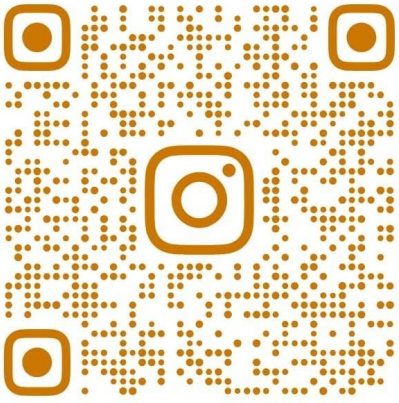
Report The Error

- If you identify any issues, writing or coding errors in this book, you are free to contact the author.
- The contact credentials are on the next section, entitled as ‘Meet The Author’.
- There are chances of the book getting republished and you being credited for the corrections…

Meet The Author

- As per 2026, the author is a male 10th Grader, pursuing his interests in AI and Electronics, sharing knowledge on the way, in an easy, free and accessible manner
- Get In Touch: (LinkedIn AND Instagram)





Topics

1. Arrays

2. Slicing

3. Arithmetic Operators

4. Broad Casting

5. Filtering

6. Aggregate/Statistical Functions

First install ***Numpy*** by,

```
"pip install numpy"
```

```
# WRITE THIS INSIDE THE TERMINAL WINDOW OF YOUR CODE EDITOR !!
```

Normal python is quite slow in terms of calculations too.

So to speed up things, we use Numpy.

★ *Arrays*

It is a superior version for storing datasets (say a list).

If we normally create a list and multiply it by 2, the element duplicate instead of doubling.

There is no way to do so!

By numpy's arrays, you can do so.

```
import numpy as np

array = np.array([1, 2, 3])
array = array * 2

print(array)
```

Result → [2, 4, 6]

★ *Multi-dimensional Arrays*

1D Arrays

When there is a single element (say A) in an array.

2D Arrays

When an array consists of a data-set (say a list like [1,2,3]).

3D Arrays

When an array contains many/multiple 2D array contents.

Eg:

```
array = np.array([[[1,2,3], [4,5,6]],
                  [[7,8,9], [10,11,12]],
                  [[13,14,15], [16,17,18]]])
```

Note: *We can create 4D, 5D, 6D, 7D, etc dimensions of an array too!*

To check dimensions of an array,

```
print(array.ndim)
```

Access the shape attribute by,

```
print(array.shape)
```

Result: (3, 2, 3) → Layers / No. of lists in a 2D list set / No. of elements in the list

To access anything i.e., element inside the 3D list we just made,

Instead of chain indexing like,

```
print(array[0][0][0])
```

we do multi-dimensional indexing,

```
print(array[0,0,0])
```

Result: 1

This is better to write & understand.

Slicing

★ *Slicing*

Whenever we want to get information of multiple lists in a pattern (like every 1st list, or every 2nd list, etc.), we use slicing.

Syntax:

```
array[start:end:step]
```

Eg:

```
import numpy as np

array = np.array([[1,2],
                  [3,4],
                  [5,6],
                  [7,8]])

print(array[0:4:1])
```

Result: [1,2], [5,6]

You can also count backwards using -ve counting.

If you write,

```
print(array[:,:])
```

The program will assume directly that start from absolute beginning & end at absolute end.

Row & Column slicing

Syntax:

```
array[start:end:steps, start:end:steps]
#           For rows      For columns
```

Task for you:

Experiment what different results you achieve by playing with the different no.

Arithmetics

★ *Arithmetics*

Scalar Arithmetic

Basically adding, subtracting, dividing, multiplying, raising contents in an array to/by a number (scalar quantity).

Functions:

```
array + 1
array - 1
array * 2
array / 2
array ** 2
```

Vectorized Math Func

Basically other operators of math on arrays.

```
numpy.sqrt(array)
numpy.round(array)
numpy.ceil(array)
numpy.floor(array)
numpy.pi
```

ceil → rounds up.
floor → rounds down.
round → rounds the number up or down by the standard round rule.

Element Wise Operators

Basically to perform arithmetic b/w specific/simply elements of two/more arrays.

```
array 1 + array 2
array 1 - array 2
```

```
array 1 * array 2
array 1 / array 2
array 1 ** array 2
```

Comparison Operators

Does what the name says, COMPARISON!

A. Greater Than(>) B. Less Than(<) C. Equal To (==) D. Mix (>= & =<)

E.g.:-

```
import numpy as np

scores = np.array([91,55,100,73])
print(scores == 100)
Result: [False, False, True, False]
```

Quite Useful!

Filtering Eg:

```
import numpy as np
scores = np.array([91,55,100,73])

scores[scores < 60]
print(scores)
```

Result: [91, 0, 100, 73]

Broadcasting

This topic will be covered later in more detail.

★ *Broadcasting*

Broadcasting allows Numpy to perform operations on arrays with different shapes by virtually expanding the dimensions of the smaller array to match the larger array's shape.

Conditions

(a) The dimensions have the same size

(OR)

(b) One of the dimensions has a size of a 1.

For Eg: If the dimensions of array1 are (4, 1) & that of array2 is (1,4), then in the first column, rule (b) is satisfied & same for column 2.

If dimension of array 1 were to be (4,1) & that of array 2 be (4,2), then broadcasting would still work.

As, for column 2, rule (a) satisfies & for column 2, rule (b) does !

Eg: Code of two compatible arrays,

```
import numpy as np

array1 = np.array([[1,2,3]])
array2 = np.array([[1],[2],[3]])

print(array1.shape)
print(array2.shape)

# We would get,
```

To be honest, you don't need to worry about this too much, as it is just a thing the Numpy does in the background.

The most basic thing to remember is that,

When two arrays are their, Say

```
[[1,2,3],       [10,10,10]
 [4,5,6]]    &
```

On arithmetic operations, Numpy treats this as,

```
[[1,2,3],       [[10,10,10],
```

```
[4,5,6]]        [10,10,10]]
```

So, the second array got virtually extended by Numpy.

Filtering

★ *Filtering*

It is termed as the process of differentiating / separating elements from an array by applying a particular condition.

Eg:

```
import numpy as np

ages = np.array([12,18,9,20,25,
                 90,65,17,60,37])

teenagers = ages[ages < 18]
adults = ages[(ages >= 18) & (ages < 65)]
seniors = ages[ages >= 65]
```

Normally, for multiple conditions in python, we use "and" operators. But, Numpy uses C type operators.

This simply means that it uses the same operators, as of C language.

So, instead of 'and', we use '&'.

If we want to use filtering method, without ruining the shape of the array, we use

```
np.where
```

following provides code,

```
adults = np.where(ages >= 18, ages, 0)
```

So, Syntax is:

```
np.where(condition, array, replace with)
```

Instead of 0, '−1' [0] np.nan can be used.

'np.nan' = not a number.

Aggregate Functions

★ *Aggregate functions*

The basic aggregation functions in the notes include:

```
np.sum(array)
np.prod(array)
np.min(array)
np.max(array)
np.mean(array)
np.median(array)
np.std(array)
```

These functions are used to get a single value from the array / summarize values.

Important: np.mean(array) gives the average.

np.median(array) gives the middle value.

np.std(array) gives standard deviation.

Random Numbers

★ *Random Numbers*

Setup

```
import numpy as np

rng = np.random.default_rng()
```

```
print(rng.integers(low=, high=, size=))
```

The 'low', 'high' & 'size' are keyword arguments, which are not necessary, but are better for readability.

They represent

(a) low → lower limit
(b) high → higher limit
(c) size → size of response & decider of dimensions.

For a sequence with a seed:

```
my_sequences = {500:[2,3,1,2]}
seed = 500
print(my_sequences[seed])

Result: [2,3,1,2]
```

For floating point numbers:

```
import numpy as np

print(np.random.uniform())
```

If brackets are empty, the range is between 0 to 1.

The syntax of the bracket is:

```
(low=, high=, size=())
```

Shuffle a random array

```
import numpy as np

rng = np.random.default_rng()
array = np.array([7,8,9,10])
rng.shuffle(array)
print(array)
```

Random Choice

Instead of,

```
rng.shuffle(array)
```

We use,

```
variable = rng.choice(array)
print(variable)
```

PYTHON

NUMPY

8 PRACTICAL PROJECTS

Complete runnable solutions • VS Code-inspired code boxes

How to use this workbook

First read the topic of the project that you wish to build, and then **IF REQUIRED**, then search only for the **CONCEPTS** required to make it.

Then, by your own perception of how the 'Writing Of The Code' process would have to be carried out, write the own **BY YOURELVES FIRST**, and then you may refer to the code solutions give below each topics.

The obvious prediction is that, you either could not write the program or you wrote it and it worked, but it was too long. So, the projects could have been executed in much less lines, but you couldn't.

So, in both those cases, the provided solutions will be helpful!

1. Number Guessing Game with Statistics

LEVEL 1 · Warm-up

Guess a randomly generated number, track your attempts, and review your guessing statistics.

```
import random
import numpy as np

secret = random.randint(1, 100)
attempts = 0
history = []

print("=== Number Guessing Game ===")
print("Guess a number between 1 and 100!")

while True:
    try:
        guess = int(input("Your guess: "))
    except ValueError:
        print("Enter a valid integer.")
        continue

    if not 1 <= guess <= 100:
        print("Choose a number from 1 to 100.")
        continue

    attempts += 1
    history.append(guess)
```

```
    if guess < secret:
        print("Too low!")
    elif guess > secret:
        print("Too high!")
    else:
        print(f"Correct! You won in {attempts} attempts.")
        break

print("\n=== Your Statistics ===")
print("Attempts:", attempts)
print("Average guess:", np.mean(history))
print("Lowest guess:", np.min(history))
print("Highest guess:", np.max(history))
print("All guesses:", history)
```

What you'll learn: while loops, conditionals, exception handling, lists, random numbers, and NumPy's mean(), min(), and max().

2. Student Marks Analyzer

LEVEL 2 · Practical NumPy

Analyze students' marks, calculate class statistics, and identify pass/fail results.

```
import numpy as np

names = ["Aarav", "Diya", "Krish", "Meera", "Rohan"]
marks = np.array([85, 92, 67, 96, 74])

print("=== Student Marks Analyzer ===")
print("Students:", names)
print("Marks:", marks)

print("\nClass average:", np.mean(marks))
print("Highest marks:", np.max(marks))
print("Lowest marks:", np.min(marks))
print("Standard deviation:", np.std(marks))

print("\n=== Individual Results ===")
for i in range(len(names)):
    result = "Pass" if marks[i] >= 33 else "Fail"
    print(f"{names[i]}: {marks[i]} marks - {result}")

print("\nStudents scoring above 80:")
print(marks[marks > 80])
```

What you'll learn: NumPy arrays, aggregate functions, Boolean masking, indexing, and standard deviation.

Note: The marks are sample data. Change the array to test different results.

3. Personal Expense Analyzer

LEVEL 3 · Data analysis

Analyze sample expenses, total spending by category, and compare spending against a budget.

```
import numpy as np

categories = np.array([
    "Food", "Transport", "Food",
    "Shopping", "Transport", "Food",
    "Entertainment"
])
expenses = np.array([150, 40, 220, 500, 60, 100, 300])

print("=== Personal Expense Analyzer ===")
print("Total spending: ₹", np.sum(expenses))
print("Average expense: ₹", np.mean(expenses))
print("Largest expense: ₹", np.max(expenses))
print("Smallest expense: ₹", np.min(expenses))

largest_index = np.argmax(expenses)
print("Biggest purchase:", categories[largest_index])

print("\nExpenses above ₹200:")
print(expenses[expenses > 200])

print("\n=== Category-wise Spending ===")
for category in np.unique(categories):
    total = np.sum(expenses[categories == category])
    print(f"{category}: ₹{total}")

budget = 1200
total = np.sum(expenses)
print("\nBudget: ₹", budget)
print("Remaining: ₹", budget - total)

if total > budget:
    print("Warning: You exceeded your budget!")
else:
    print("You stayed within your budget.")
```

What you'll learn: sum(), mean(), argmax(), unique(), Boolean indexing, and combining two related arrays.

Note: All expenses are fictional sample values. Negative remaining money means the budget was exceeded.

4. Random Dice Simulator

LEVEL 2 · Random simulation

Simulate many dice rolls and calculate how frequently each face appears.

```
import numpy as np

rng = np.random.default_rng()
rolls_count = int(input("How many times should we roll? "))

if rolls_count <= 0:
    print("Enter a positive number.")
else:
    rolls = rng.integers(1, 7, size=rolls_count)

    print("\nFirst 20 rolls:")
    print(rolls[:20])

    print("\n=== Dice Statistics ===")
    for face in range(1, 7):
        count = np.sum(rolls == face)
        percentage = count / rolls_count * 100
        print(
            f"Face {face}: {count} times "
            f"({percentage:.2f}%)"
        )

    print("\nAverage roll:", np.mean(rolls))
    print("Minimum roll:", np.min(rolls))
    print("Maximum roll:", np.max(rolls))
```

What you'll learn: NumPy random generators, integers(), array comparisons, counting with sum(), and simulation statistics.

Note: The upper bound in rng.integers(1, 7) is excluded, so results range from 1 to 6.

5. Matrix Calculator

LEVEL 3 · Array manipulation

Perform common operations on two matrices and explore matrix multiplication and inverses.

```
import numpy as np

A = np.array([
    [1, 2],
    [3, 4]
])
B = np.array([
    [5, 6],
    [7, 8]
])

print("Matrix A:\n", A)
print("\nMatrix B:\n", B)
print("\nAddition:\n", A + B)
print("\nSubtraction:\n", A - B)
print("\nElement-wise multiplication:\n", A * B)
print("\nMatrix multiplication:\n", A @ B)
print("\nTranspose of A:\n", A.T)
print("\nDeterminant of A:", np.linalg.det(A))

try:
    print("\nInverse of A:\n", np.linalg.inv(A))
except np.linalg.LinAlgError:
    print("This matrix has no inverse.")
```

What you'll learn: Two-dimensional arrays, transpose, determinants, inverses, and linear algebra.

Note: A * B multiplies corresponding elements; A @ B performs matrix multiplication.

6. Student Ranking System

LEVEL 3 · Sorting and indexing

Rank students from highest to lowest marks while keeping each name matched to the correct score.

```
import numpy as np

names = np.array(["Aarav", "Diya", "Krish", "Meera", "Rohan"])
marks = np.array([85, 92, 67, 96, 74])

ranking = np.argsort(marks)[::-1]

print("=== Student Rankings ===")
for rank, index in enumerate(ranking, start=1):
    print(f"{rank}. {names[index]} - {marks[index]} marks")

top_index = np.argmax(marks)
print("\nTopper:", names[top_index])
print("Topper's marks:", marks[top_index])

print("\nStudents scoring at least 80:")
print(names[marks >= 80])

print("\nTop three students:")
for index in ranking[:3]:
    print(names[index], marks[index])
```

What you'll learn: argsort(), reverse slicing, argmax(), Boolean masks, and index-based sorting.

Note: argsort() returns indices in ascending order; [::-1] reverses them to descending order.

7. NumPy Weather Analyzer

LEVEL 2 · Statistical analysis

Analyze a week's temperatures and find the hottest day, coldest day, and days above average.

```
import numpy as np

days = np.array(["Mon", "Tue", "Wed", "Thu", "Fri", "Sat", "Sun"])
temperatures = np.array([32, 35, 31, 37, 34, 39, 33])

average = np.mean(temperatures)

print("=== Weekly Weather Report ===")
print("Average temperature:", average)
print("Hottest temperature:", np.max(temperatures))
print("Coldest temperature:", np.min(temperatures))

hot_index = np.argmax(temperatures)
cold_index = np.argmin(temperatures)
print("Hottest day:", days[hot_index])
print("Coldest day:", days[cold_index])

print("\nDays above average:")
print(days[temperatures > average])

print("\nTemperature differences from average:")
print(temperatures - average)
print("\nTemperature standard deviation:")
print(np.std(temperatures))
```

What you'll learn: argmax(), argmin(), Boolean indexing, array arithmetic, and standard deviation.

Note: Temperatures are fictional sample data. The subtraction from average is applied to every array element.

8. NumPy Array Data Cleaner

LEVEL 3 · Data cleaning

Filter out missing-measurement markers, calculate the mean of valid values, and replace missing entries.

```
import numpy as np

# -1 represents a missing measurement
data = np.array([25, 28, -1, 30, 27, -1, 32, 29])
print("Original data:", data)

valid = data != -1
valid_values = data[valid]

if len(valid_values) == 0:
    print("No valid measurements available.")
else:
    mean_value = np.mean(valid_values)
    cleaned = np.where(valid, data, mean_value)

    print("Valid measurements:", valid_values)
    print("Mean:", mean_value)
    print("Cleaned data:", cleaned)
    print("Minimum:", np.min(cleaned))
    print("Maximum:", np.max(cleaned))
    print("Standard deviation:", np.std(cleaned))
```

What you'll learn: Boolean masks, np.where(), filtering, and handling missing measurements.

Note: Here -1 is a chosen placeholder. Real datasets may use NaN or another missing-value convention.
